Animation in React

Animations have the powerful ability to turn a boring, static application into a more dynamic, memorable experience for users.

Consider :

1. The CSS properties want to animate
2. The duration over which the animation to be performed
3. An easing function that determines how the animation progresses through the duration of each cycle

CSS Animation:

```
// App.js
import React from 'react';
import './App.css';

function App() {
  return (
    <div className="App">
      <div className="box">Animate me!</div>
    </div>
  );
}
export default App;
```

```
/* App.css */
.box {
  width: 100px;
  height: 100px;
  background-color: skyblue;
  transition: transform 0.5s;
}
```

```
.box:hover {
  transform: translateX(100px);
}
```

2.React Transition Group:

This library provides components for managing animations when elements enter or leave the DOM.

npm install react-transition-group

```
// App.js
import React, { useState } from 'react';
import { CSSTransition } from 'react-transition-group';
import './App.css';

function App() {
  const [show, setShow] = useState(false);

  return (
    <div className="App">
      <button onClick={() => setShow(!show)}>Toggle</button>
      <CSSTransition
        in={show}
        timeout={300}
        classNames="fade"
        unmountOnExit
      >
        <div className="box">Animate me!</div>
      </CSSTransition>
    </div>
  );
}
```

```
export default App;
```

```
/* App.css */
.fade-enter {
  opacity: 0;
}
.fade-enter-active {
  opacity: 1;
  transition: opacity 300ms;
}
.fade-exit {
  opacity: 1;
}
.fade-exit-active {
  opacity: 0;
  transition: opacity 300ms;
}
```

3rd way to animate: Frame motion:

Framer Motion is a powerful library for animations in React.

npm install framer-motion

```
// App.js
import React, { useState } from 'react';
import { motion } from 'framer-motion';

function App() {
  const [show, setShow] = useState(false);

  return (
    <div className="App">
      <button onClick={() => setShow(!show)}>Toggle</button>
      {show && (
        <motion.div
```

```
        initial={{ opacity: 0 }}
        animate={{ opacity: 1 }}
        exit={{ opacity: 0 }}
        transition={{ duration: 0.5 }}
        className="box"
      >
        Animate me!
      </motion.div>
    )}
  </div>
 );
}

export default App;
```

4: React Spring:

React Spring is another library for physics-based animations.

npm install @react-spring/web

```
// App.js
import React, { useState } from 'react';
import { useSpring, animated } from '@react-spring/web';

function App() {
 const [show, setShow] = useState(false);

 const props = useSpring({
  opacity: show ? 1 : 0,
  transform: show ? 'translateX(0px)' : 'translateX(-100px)',
  config: { duration: 500 },
 });

 return (
  <div className="App">
```

```
      <button onClick={() => setShow(!show)}>Toggle</button>
      <animated.div style={props} className="box">
        Animate me!
      </animated.div>
    </div>
  );
}

export default App;
```

.

5.. AutoAnimate:

How AutoAnimate Works

autoAnimate takes one argument: a reference to the parent element which you would like to animate.

The way the library works is that the parent element will be "auto animated" along with any of its immediate children.

Animations take place whenever one of three events occurs to this parent element: If a child element is **added**, **removed**, or **moved** around.

We're going to look at how you can use AutoAnimate with three examples: an expandable component, list component and a grid component.

How to Use AutoAnimate

There are two steps to start using auto animate:

1. Install it in your project using either yarn or NPM

```
npm install @formkit/auto-animate
```

Import the auto animate function from the library itself

```
import autoAnimate from '@formkit/auto-animate'
```

To animate any parent element, pass a reference of the element to the function.

```
import { useEffect, useRef } from 'react'

function Component() {
```

```
  const parentRef = useRef(null)

  useEffect(() => {
    if (parentRef.current) {
      autoAnimate(parentRef.current);
    }
  }, [parent])

  return (
    <div ref={parentRef}>
    // ...
  )
}
```

Click on a div and expand it to show some more text.

First, we create a div with some text to display in its initial state as well as some text to reveal when clicked.

To animate the text opening, we use the useRef hook to reference the parent element and then pass that reference to the auto animate function.

How to Animate Lists with AutoAnimate

Another great use case for auto animate is with a list component.

Animate new items that are added to the list.

```
import { useState } from "react"
import autoAnimate from "@formkit/auto-animate"

export default function App() {
  const [items, setItems] = useState(["Buy Gas", "Do Laundry"])

  function addItem() {
    const item = "Go To Store"
    setItems([...items, item])
  }
```

```
  return (
    <>
      <ul ref={parent}>
        {items.map((item) => (
          <li key={item}>{item}</li>
        ))}
      </ul>
      <button onClick={addItem}>Add Todo</button>
    </>
  )
}
```